



Remote

Method

Invocation

Library Management System



Presentation *Plan*





Our project is a **Java console application** that implements a **remote library management system**. It allows the user to interact with library services through a simple command-line interface.



The application follows a **distributed model**, meaning the different parts of the system can operate across separate physical machines, ensuring that our end user can manage library operations reliably from any remote location.



The system is built using **Java RMI (Remote Method Invocation)**, a technology that enables remote communication between programs written in Java. With RMI, users can perform actions on a remote server just as if they were executing them locally.





System Functionalities



****User Authentication****

- Our end user must log in using a valid username and password.
- Only after authentication, our user can interact with the system.



****Book Management****

- **Add a Book:** Input title, author, and ISBN .
- **Get Book Info:** Search for a book using its ISBN and display its details.
- **Update Availability:** Modify the availability status of a book (true or false).
- **Update Book Information:** Modify a book's title or author using its ISBN.
- **Delete Book:** Permanently remove a book using its ISBN.
- **List All Books:** View a complete list of books with their informations.



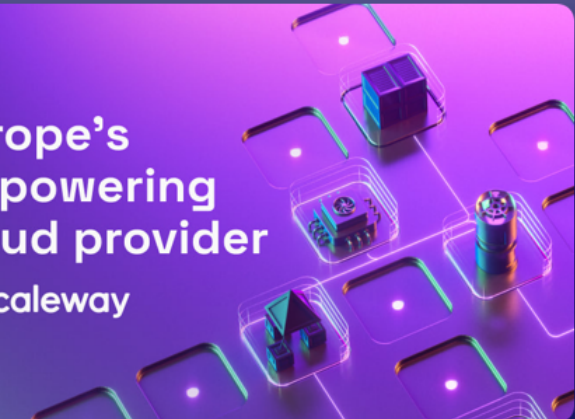
Java RMI is the core technology that enables our distributed system to function. It allows the client application to invoke methods on objects located remotely on the server

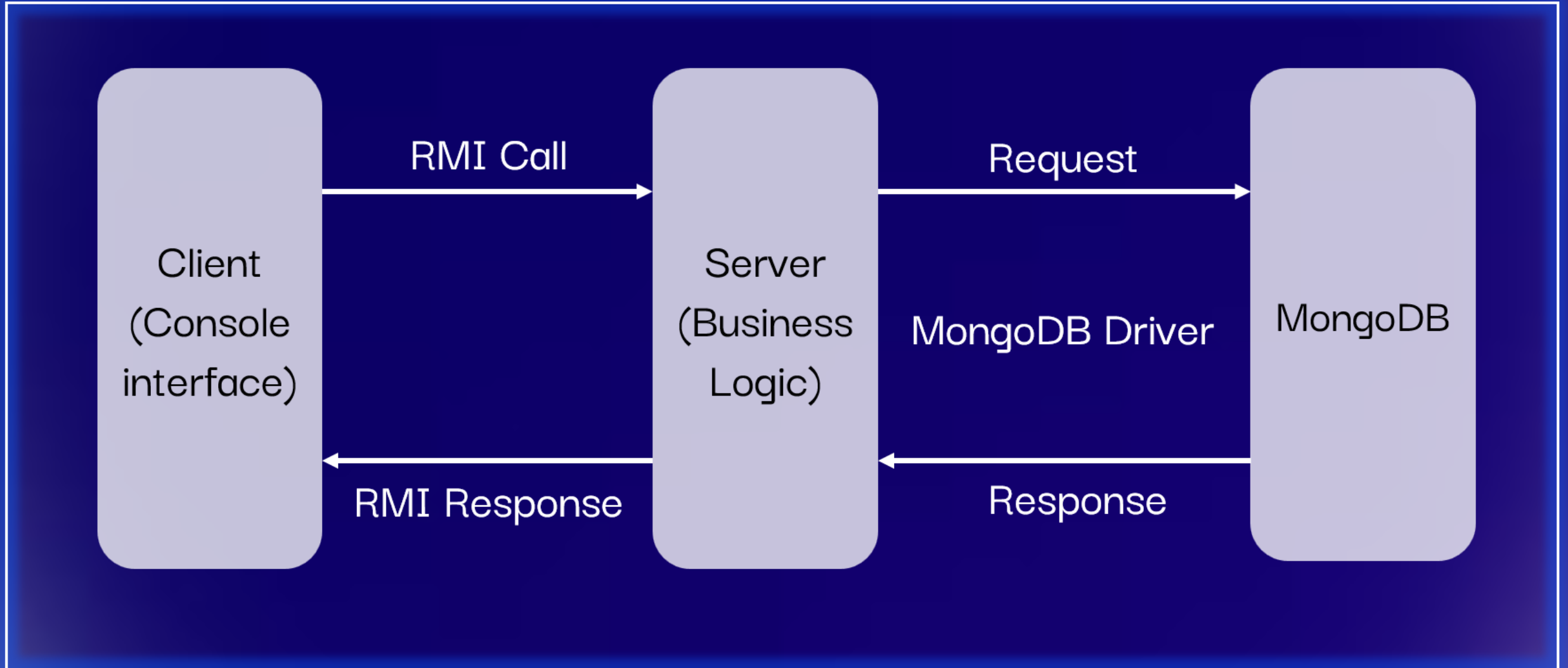


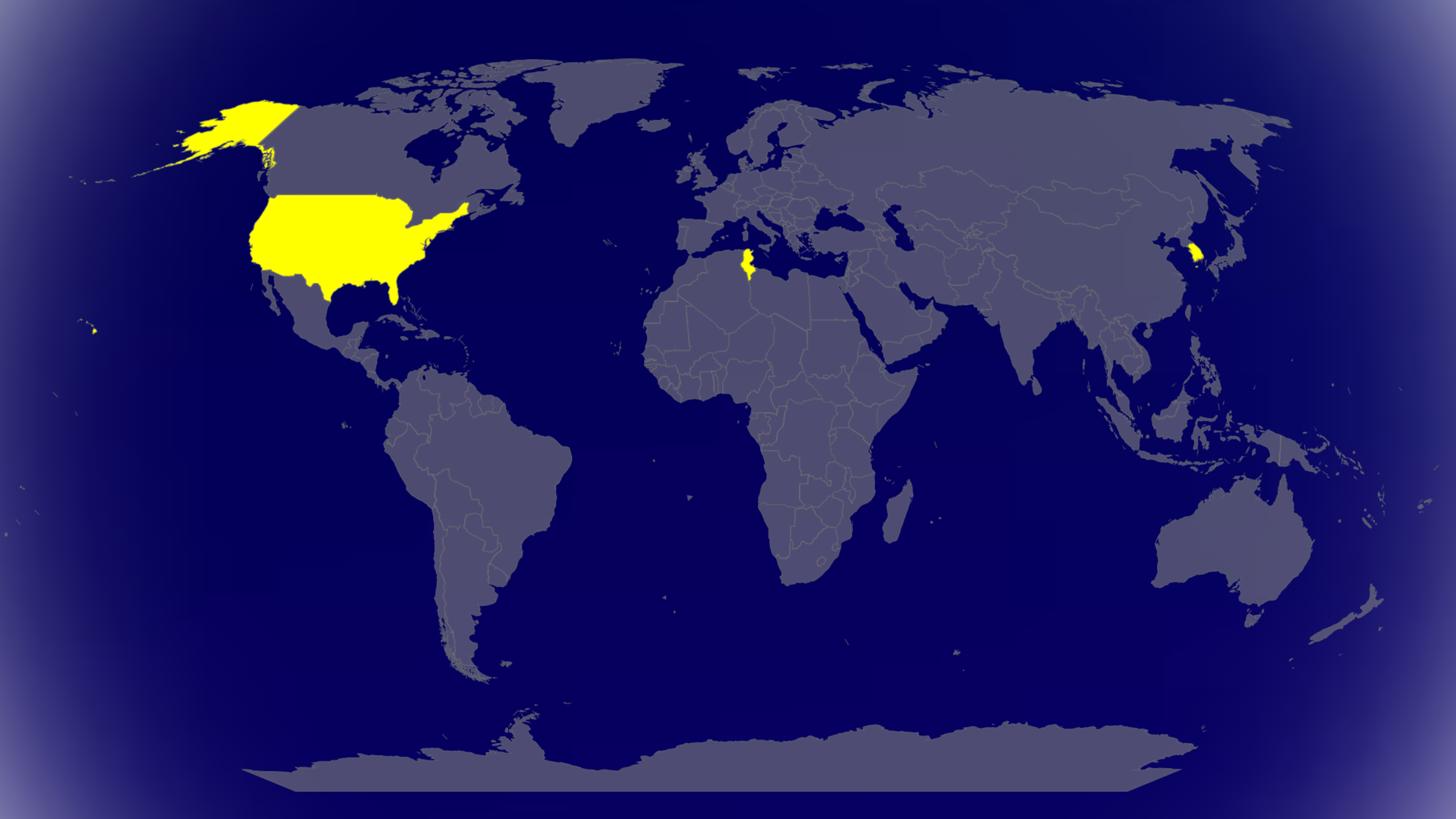
MongoDB Cloud is a NoSQL cloud-based database service where all user and book data are stored. It offers flexible document-based storage.

Scaleway is an IaaS cloud provider. It enables remote access across 38 regions by offering dedicated machines with public IP address, making them reachable all across the world. By deploying our java RMI application on a cloud remote machine, we ensure that the user can connect from anywhere with internet access.

Europe's
empowering
cloud provider
 Scaleway





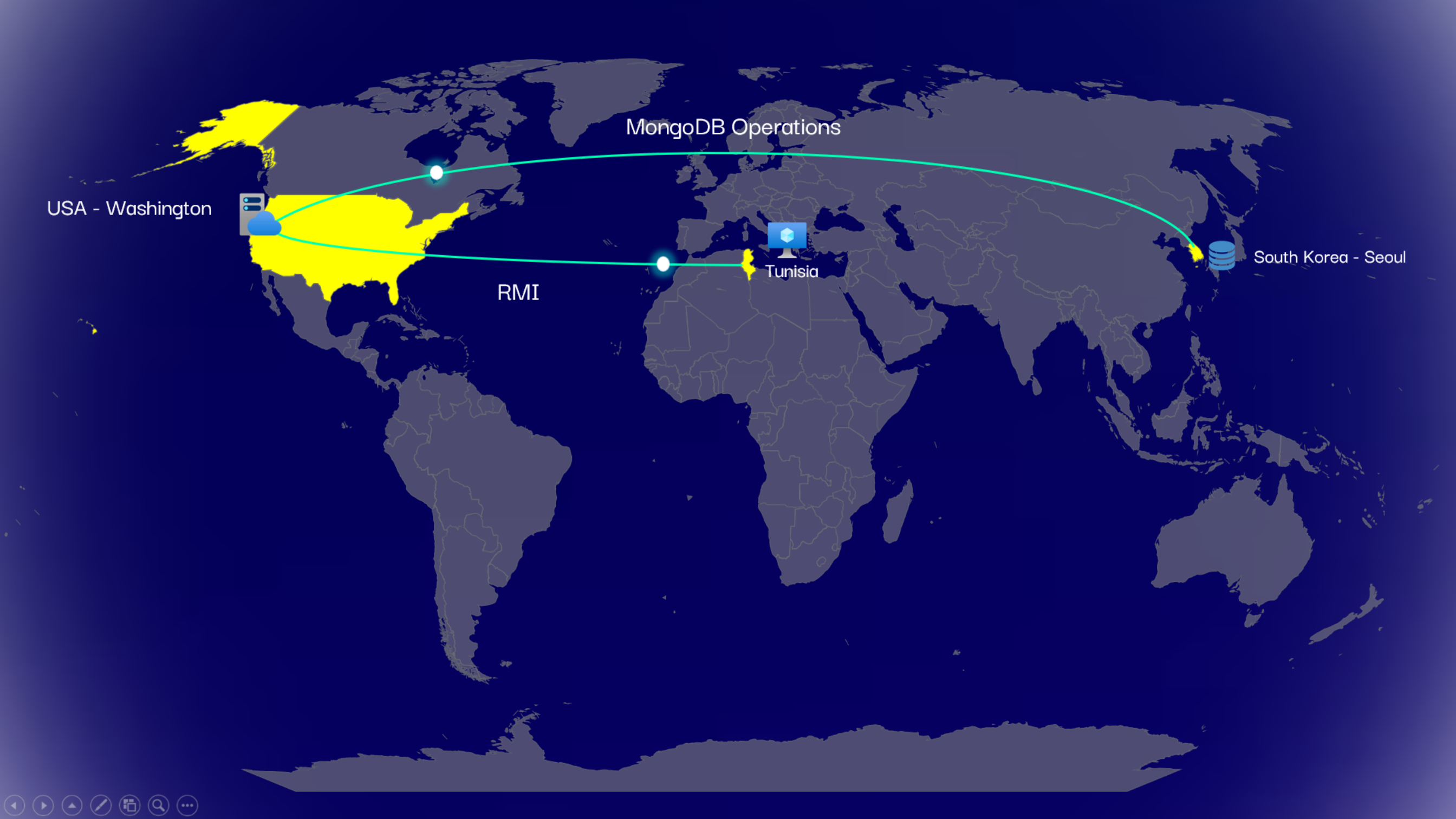


- 
- Server Machine : (USA,Washington)
 - IP: 20.64.147.176
 - 201-245 S Alder St, Moses Lake, WA 98837
 - Provider: Scaleway



Client Machine : (Tunisia)

- 
- A map of Asia is shown in a dark blue color. The country of South Korea is highlighted in a bright yellow color. A yellow line originates from the highlighted South Korea and extends diagonally down and to the right, then turns horizontally to the right, pointing towards a list of specifications.
- Dedicated MongoDB Cluster : (South Korea, Seoul)
 - Provider: AWS
 - 2 GB Ram
 - 2 vCPU
 - 10 GB NVME M.2



MongoDB Operations

USA - Washington

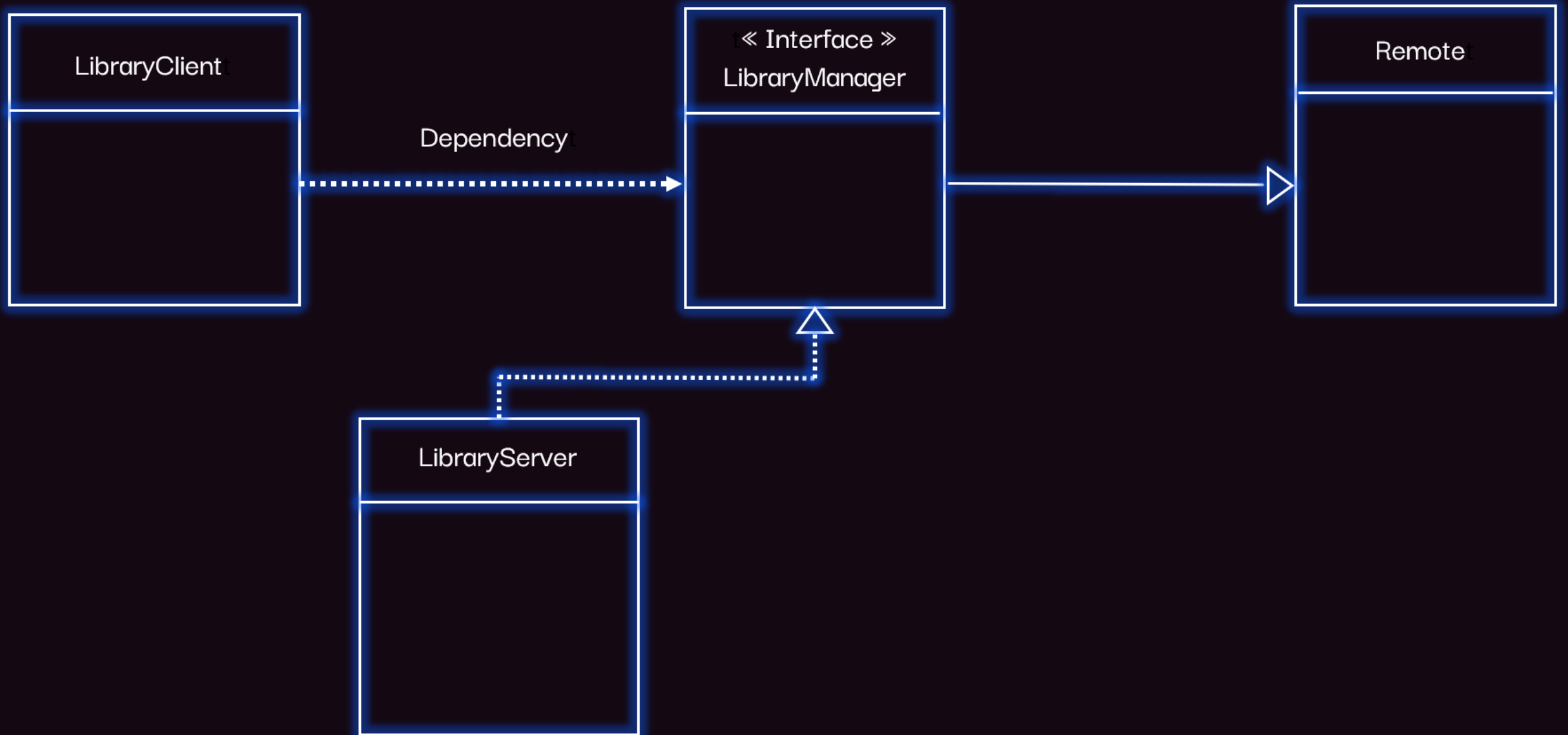
South Korea - Seoul

Tunisia

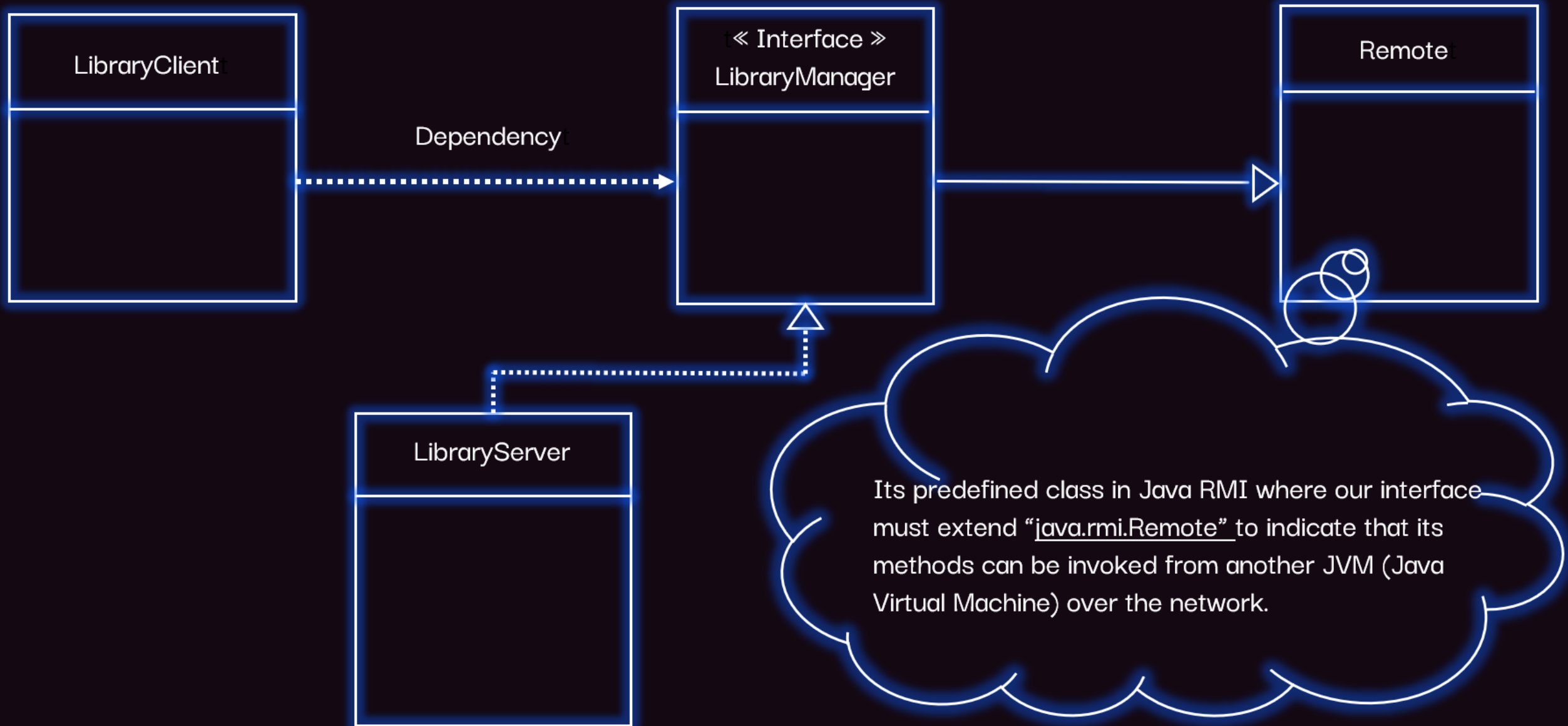
RMI

Source Code Explanation

Code Structure



Code Structure



« Interface » Library Manager

Because remote communication over the network can fail java requires these exceptions to be handled:

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface LibraryManager extends Remote { 2 usages  
    boolean loginUser(String username, String password) throws RemoteException; 1 usage  
  
    boolean addBook(String title, String author, String ISBN) throws RemoteException; 1 usage  
    String getBook(String ISBN) throws RemoteException; 1 usage  
    boolean updateBookAvailability(String ISBN, boolean availability) throws RemoteException; no usages  
    boolean removeBook(String ISBN) throws RemoteException; 1 usage  
    String listBooks() throws RemoteException; 1 usage  
    boolean updateBookDetails(String ISBN, String newTitle, String newAuthor) throws RemoteException; 1 usage  
}
```

LibraryServer Class

```

import org.bson.Document;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class LibraryServer implements LibraryManager {

    private final MongoClient mongoClient;
    private final MongoCollection<Document> usersCollection;
    private final MongoCollection<Document> booksCollection;

    public LibraryServer() {
        MongoClient mongoClient = MongoClient.create("mongodb+srv://moezkr:moezkr@rmicluste.b9vwsfv.mongodb.net/");
        MongoDatabase database = mongoClient.getDatabase("libraryDB");
        usersCollection = database.getCollection("users");
        booksCollection = database.getCollection("books");
    }

    @Override
    public boolean loginUser(String username, String password) throws RemoteException {
        Document user = usersCollection.find(Filters.eq("username", username)).first();
        if (user == null) return false;
        return user.getString("password").equals(password);
    }

    @Override
    public boolean addBook(String title, String author, String ISBN) throws RemoteException {
        Document existing = booksCollection.find(Filters.eq("ISBN", ISBN)).first();
        if (existing != null) {
            System.out.println("⚠ Book already exists: " + ISBN);
            return false;
        }

        Document book = new Document("title", title)
            .append("author", author)
            .append("ISBN", ISBN)
            .append("available", true);
        booksCollection.insertOne(book);
        System.out.println("📖 Book added: " + book.toJson());
        return true;
    }

    public String getBook(String ISBN) throws RemoteException {
        Document book = booksCollection.find(Filters.eq("ISBN", ISBN)).first();

        if (book != null) {
            String title = book.getString("title");
            String author = book.getString("author");
            String isbn = book.getString("ISBN");
            boolean available = book.getBoolean("available");

            return String.format("📖 Title: %s | 📑 Author: %s | 📖 ISBN: %s | 🟢 Available: %b",
                title, author, isbn, available);
        } else {
            return "🔍 Book not found.";
        }
    }
}

```

Initializes the connection to MongoDB cloud cluster and sets up access to two collections: users for login info and books for library data.

```

@Override 1 usage
public boolean updateBookAvailability(String ISBN, boolean availability) throws RemoteException {
    Document book = booksCollection.find(Filters.eq( fieldName: "ISBN", ISBN)).first();

    if (book != null) {
        booksCollection.updateOne(Filters.eq( fieldName: "ISBN", ISBN),
            new Document("$set", new Document("available", availability)));
        System.out.println("🔵 Updated availability for ISBN: " + ISBN);
        return true;
    } else {
        System.out.println("❌ Book with ISBN " + ISBN + " not found.");
        return false;
    }
}

```

Finds a book by ISBN and updates its availability (true/false). Returns whether the update was successful.

```

@Override 1 usage
public boolean updateBookDetails(String ISBN, String newTitle, String newAuthor) throws RemoteException {
    Document existingBook = booksCollection.find(Filters.eq( fieldName: "ISBN", ISBN)).first();
    if (existingBook == null) {
        System.out.println("❌ Book with ISBN " + ISBN + " not found.");
        return false;
    }

    Document updateFields = new Document();
    if (newTitle != null && !newTitle.isEmpty()) {
        updateFields.append("title", newTitle);
    }

    if (newAuthor != null && !newAuthor.isEmpty()) {
        updateFields.append("author", newAuthor);
    }

    if (updateFields.isEmpty()) {
        System.out.println("⚠️ No new details provided for update.");
        return false;
    }

    booksCollection.updateOne(Filters.eq( fieldName: "ISBN", ISBN), new Document("$set", updateFields));
    System.out.println("🔵 Book details updated for ISBN: " + ISBN);
    return true;
}

```

Updates the title and/or author of a book based on the provided values, keeping any fields unchanged if left empty. Returns true if update succeeds.

```

@Override 1 usage
public boolean removeBook(String ISBN) throws RemoteException {
    // Check if the book exists
    boolean bookExists = booksCollection.countDocuments(Filters.eq("ISBN", ISBN)) > 0;

    if (bookExists) {
        // Book exists, perform deletion
        booksCollection.deleteOne(Filters.eq("ISBN", ISBN));
        System.out.println("📖 Book removed with ISBN: " + ISBN);
        return true;
    } else {
        // Book does not exist
        System.out.println("📖 Book with ISBN " + ISBN + " not found.");
        return false;
    }
}

@Override 1 usage
public String listBooks() throws RemoteException {
    StringBuilder sb = new StringBuilder();
    FindIterable<Document> allBooks = booksCollection.find();
    for (Document doc : allBooks) {
        sb.append("📖 Title: ").append(doc.getString("title"))
          .append(" | 📖 Author: ").append(doc.getString("author"))
          .append(" | 📖 ISBN: ").append(doc.getString("ISBN"))
          .append(" | 📖 Available: ").append(doc.getBoolean("available"))
          .append("\n");
    }

    if (sb.length() > 0) {
        return sb.toString();
    } else {
        return "📖 No books found.";
    }
}

public static void main(String[] args) {
    try {
        System.setProperty("java.rmi.server.hostname", "20.64.147.176");
        LibraryServer obj = new LibraryServer();
        LibraryManager stub = (LibraryManager) UnicastRemoteObject.exportObject(obj, 5002);
        Registry registry = LocateRegistry.createRegistry(5000);
        registry.rebind("LibraryManager", stub);
        System.out.println("🟢 Library Server is running at 20.64.147.176:5000");

        synchronized (LibraryServer.class) {
            LibraryServer.class.wait(); // keep server alive
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

System.setProperty("java.rmi.server.hostname", "20.64.147.176");

By default, the server will use localhost in the stub it gives to clients. This works only if the client is on the same machine. If our case the server is hosted in a cloud and clients are connecting from other machines they will not be able to reach localhost. So we explicitly set the public IP address to ensure clients can connect to the server correctly.

LocateRegistry.createRegistry(5000);

Creates an RMI registry on **port 5000** so clients can lookup remote objects.
Clients first connect here to retrieve the stub proxy.

UnicastRemoteObject.exportObject(obj, 5002);

Makes the object ready to receive remote method calls and exports it to listen on **port 5002**.
This port handles all remote method calls from clients.

LibraryClient Class

```

import java.rmi.Naming;
import java.util.Scanner;
public class LibraryClient {
    public static void main(String[] args) {
        try {
            Scanner scanner = new Scanner(System.in);
            System.out.println("Connecting to server...");
            LibraryManager stub = (LibraryManager) Naming.lookup("rmi://20.64.147.176:5000/LibraryManager");
            System.out.println("✅ Connected to Library System");

            System.out.print("Enter username: ");
            String username = scanner.nextLine();
            System.out.print("Enter password: ");
            String password = scanner.nextLine();

            if (!stub.loginUser(username, password)) {
                System.out.println("❌ Invalid login !!!");
                return;
            }

            boolean running = true;
            while (running) {
                System.out.println("\n📖 Menu:");
                System.out.println("1. Add a book");
                System.out.println("2. List books");
                System.out.println("3. Remove a book");
                System.out.println("4. Update book availability");
                System.out.println("5. Get book by ISBN");
                System.out.println("6. Update book details (title/author)");
                System.out.println("7. Exit");
                System.out.print("Enter choice: ");
                int choice = scanner.nextInt();
                scanner.nextLine();

                switch (choice) {
                    case 1:
                        System.out.print("Title: ");
                        String title = scanner.nextLine();
                        System.out.print("Author: ");
                        String author = scanner.nextLine();
                        System.out.print("ISBN: ");
                        String isbn = scanner.nextLine();
                        boolean added = stub.addBook(title, author, isbn);
                        if (added) {
                            System.out.println("✅ Book added successfully!");
                        } else {
                            System.out.println("⚠️ Book with this ISBN already exists.");
                        }
                        break;

                    case 2:
                        System.out.println(stub.listBooks());
                        break;
                }
            }
        }
    }
}

```

This line connects the client to the remote RMI server. It uses Naming.lookup to search for the remote object named "LibraryManager" at the IP address 20.64.147.176 on port 5000.

```
case 3:
    System.out.print("ISBN to remove: ");
    String removeISBN = scanner.nextLine();

    boolean isRemoved = stub.removeBook(removeISBN);

    if (isRemoved) {
        System.out.println("🗑️ Book removed successfully.");
    } else {
        System.out.println("🔍 Book with ISBN " + removeISBN + " not found.");
    }
    break;

case 4:
    System.out.print("ISBN: ");
    String upISBN = scanner.nextLine();

    System.out.print("Available (true/false): ");
    String input = scanner.nextLine().trim().toLowerCase();

    if (!input.equals("true") && !input.equals("false")) {
        System.out.println("❗ Invalid input. Please enter 'true' or 'false'.");
        break;
    }

    boolean available = Boolean.parseBoolean(input);
    boolean updated_availability = stub.updateBookAvailability(upISBN, available);

    if (updated_availability) {
        System.out.println("✅ Book availability updated successfully.");
    } else {
        System.out.println("❌ Book with ISBN " + upISBN + " not found.");
    }
    break;

case 5:
    System.out.print("ISBN: ");
    String searchISBN = scanner.nextLine();
    System.out.println(stub.getBook(searchISBN));
    break;
```

```

        case 6:
            System.out.print("ISBN of book to update: ");
            String updateISBN = scanner.nextLine();
            System.out.print("New title (leave blank to keep unchanged): ");
            String newTitle = scanner.nextLine();
            System.out.print("New author (leave blank to keep unchanged): ");
            String newAuthor = scanner.nextLine();
            boolean updated = stub.updateBookDetails(updateISBN, newTitle, newAuthor);
            if (updated) {
                System.out.println("✅ Book updated.");
            } else {
                System.out.println("⚠️ Book not found or no changes provided.");
            }
            break;

        case 7:
            running = false;
            break;

        default:
            System.out.println("❌ Invalid choice");
    }
}

} catch (Exception e) {
    System.out.println("❌ error connection");
}

}

}

```

Official References

These resources provide in-depth information on how Java RMI works, including the roles of stubs and skeletons, the process of exporting remote objects, and the underlying communication mechanisms.

Detailed technical specifications of the RMI architecture, including information on stubs, skeletons, and the RMI transport layer.

<https://docs.oracle.com/javase/8/docs/platform/rmi/spec/rmiTOC.html>

Official documentation for the `UnicastRemoteObject` class, which is used in our project to export remote objects for RMI.

<https://docs.oracle.com/javase/8/docs/api/java/rmi/server/UnicastRemoteObject.html>

Official documentation for `java.rmi.RemoteException`:

https://docs.oracle.com/javase/8/docs/api/java/rmi/RemoteException.html?utm_source=chatgpt.com

Execution Scenario